

“**self-supervised** ways to learn representations of language, instead of creating representations by hand via **feature engineering**”

Static Embedding

Maryam Abdolali
KNTU, Fall 2025

Vector Embedding

- **vector semantics:** represent a word as a point in a multidimensional semantic space that is derived from the distributions of word neighbors.
- Vectors for representing words are called **embeddings**.
- **Goal:** similar words to be near each other in dense high-dimensional space



Motto: you shall know a word by the company it keeps

Simple count-based embeddings

- **First way** to compute word vector embeddings: **co-occurrence matrix**
- A way of representing how often words co-occur.
- Particular kind of co-occurrence matrix, the **word-context matrix**:
 - dimensionality $|V| \times |V|$
 - each cell records the number of times the row (target) word and the column (context) word co-occur nearby in some training corpus.
- What do we mean by ‘**nearby**’?
 - A context window around the word, e.g., *4 words to the left and 4 words to the right*. If we do that, each cell will represent the number of times (in some training corpus) the column word occurs in such a ± 4 word window around the row word

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Example

- Window length 1 (more common: 5–10)
- Symmetric (irrelevant whether left or right context)

- Example corpus:

- *I like deep learning*
- *I like NLP*
- *I enjoy flying*

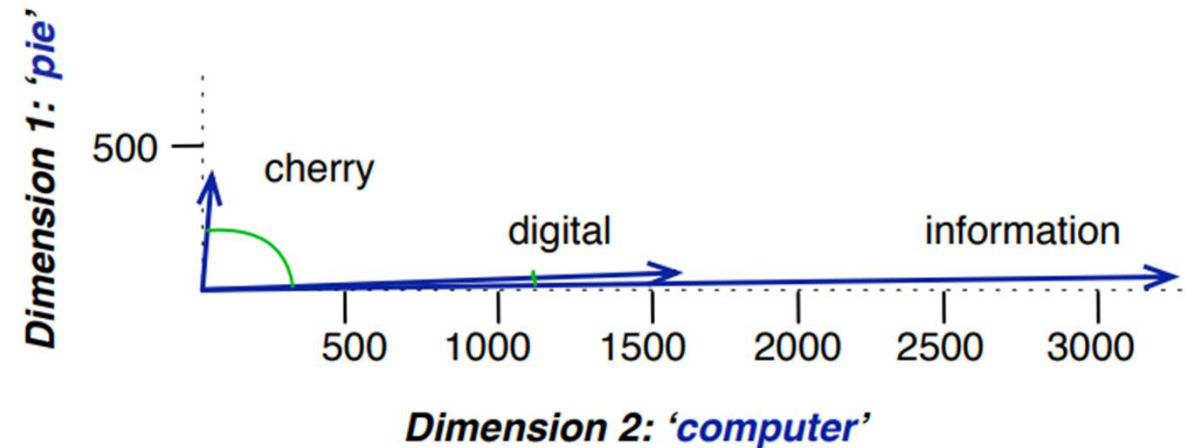
counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

So?

- Simple count co-occurrence vectors
 - Vectors increase in **size** with vocabulary
 - Very high dimensional: require a lot of storage (though sparse)
 - Subsequent classification models have sparsity issues, Models are less robust
- **Low-dimensional vectors**
 - **Idea:** store “most” of the important information in a fixed, small number of dimensions: a dense vector
 - Usually 25–1000 dimensions
 - **How to reduce the dimensionality? SVD**
 - Running an SVD on raw counts doesn’t work well
 - Scaling the counts in the cells can help a lot
 - function words (the, he, has) are too frequent -> syntax has too much impact.
 - Some fixes: 1. log the frequencies 2. $\min(X, t)$, with $t \approx 100$ 3. Ignore the function words

Cosine for measuring similarity

- Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for visualization purposes).
- The vector for digital is outlined in red

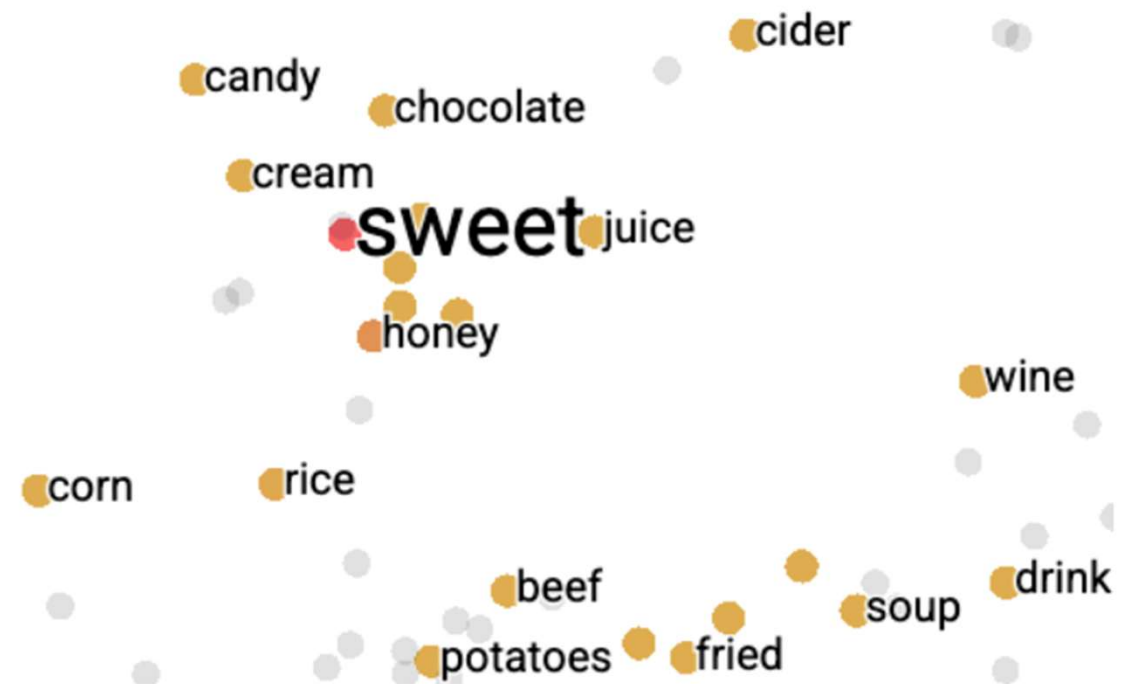


- A (rough) graphical demonstration of cosine similarity, showing vectors for three words (cherry, digital, and information)

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Word2vec

- we saw how to **represent a word** as a **sparse, long vector** with dimensions corresponding to number of words in the vocabulary.
- We now introduce a more powerful word representation:
 - embeddings, **short dense vectors**
- Word2vec a framework for learning word vectors (Mikolov et al. 2013)
- Based on simple **neural networks**



There are two models for Word2Vec

Continuous Bag Of Words (CBOW) and **Skip-Gram**. While **Skip-Gram** model predicts context words given a center word, **CBOW** model predicts a center word given context words.

Skip-gram: works well with **small amount of the training data**, represents well even **rare** words or phrases

CBOW: several times **faster to train** than the skip-gram, slightly better accuracy for the frequent words

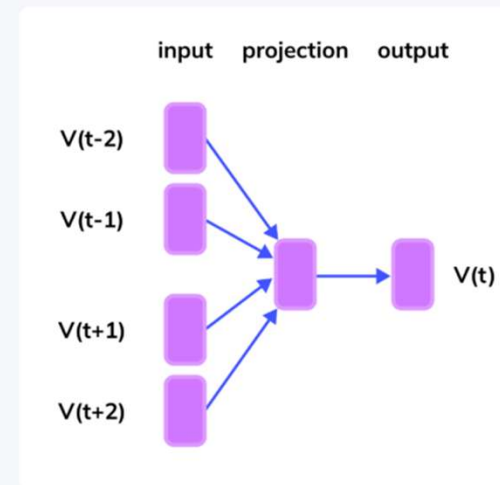
Skip-Gram model is a better choice most of the time due to its ability to predict **infrequent words**, but this comes at the price of increased computational cost.

If **training time is a big concern**, and you have **large enough data** to overcome the issue of predicting infrequent words, **CBOW** model may be a more viable choice.

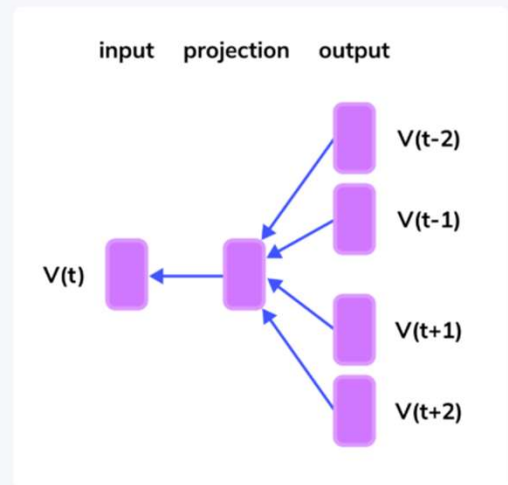
Jay was hit by a red bus in...

by	a	red	bus	in
----	---	-----	-----	----

CBOW



Skip-gram



Different models different datasets for train

• Continuous Bag of Words

- The goal is to be able to use the context surrounding a particular word and predict the **target word**.

Source Text	Training Samples			
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. →	The	quick	brown	(the, quick) (the, brown)
The	quick	brown		
The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. →	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)
quick	brown	fox		
The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. →	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
brown	fox	jumps		
The quick brown <table><tr><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. →	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
fox	jumps	over		

Source Text

Training Samples
(context, target)

The

quick

brown

fox jumps over the lazy dog.

→

((quick, brown), The)

The

quick

brown

fox

jumps over the lazy dog.

→

((The, brown, fox), quick)

The

quick

brown

fox

jumps

over the lazy dog.

→

((The, quick, fox, jumps), brown)

The

quick

brown

fox

jumps

over

the lazy dog.

→

((quick, brown, jumps, over), fox)

• Skipgram

- The goal here is to be able to predict the **surrounding words** of a particular input word. The input word is the word in the center of the context while the output of the neural network is all the surrounding words in the context being predicted.

Skipgram

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a



Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

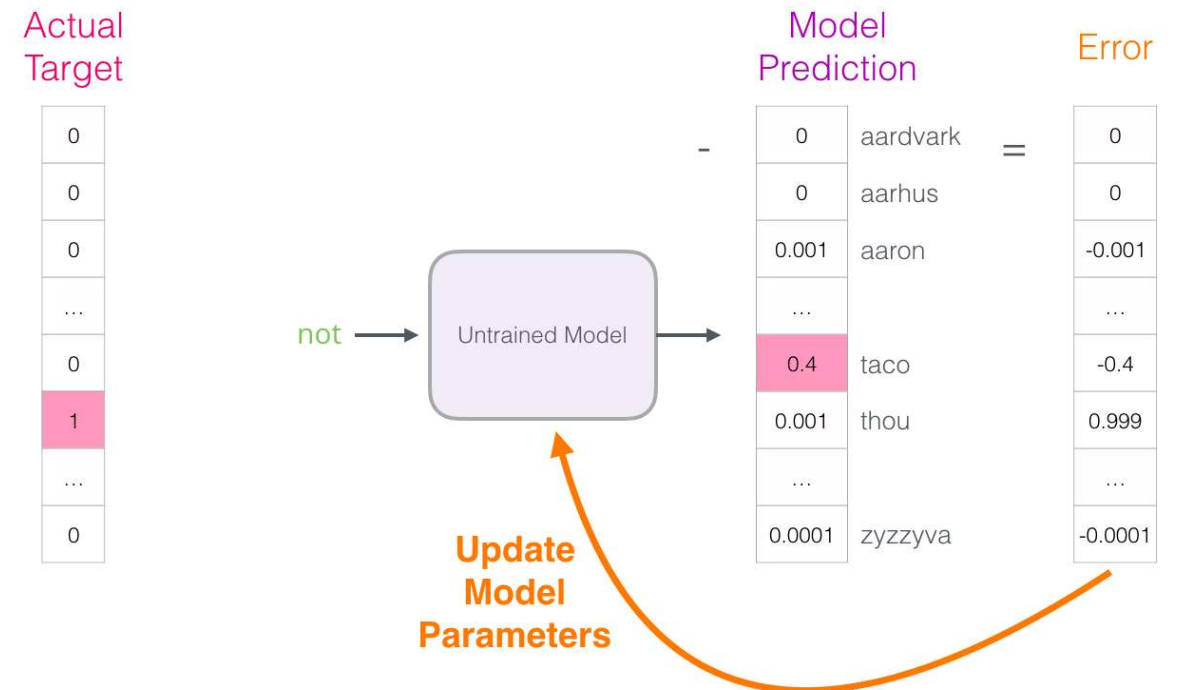
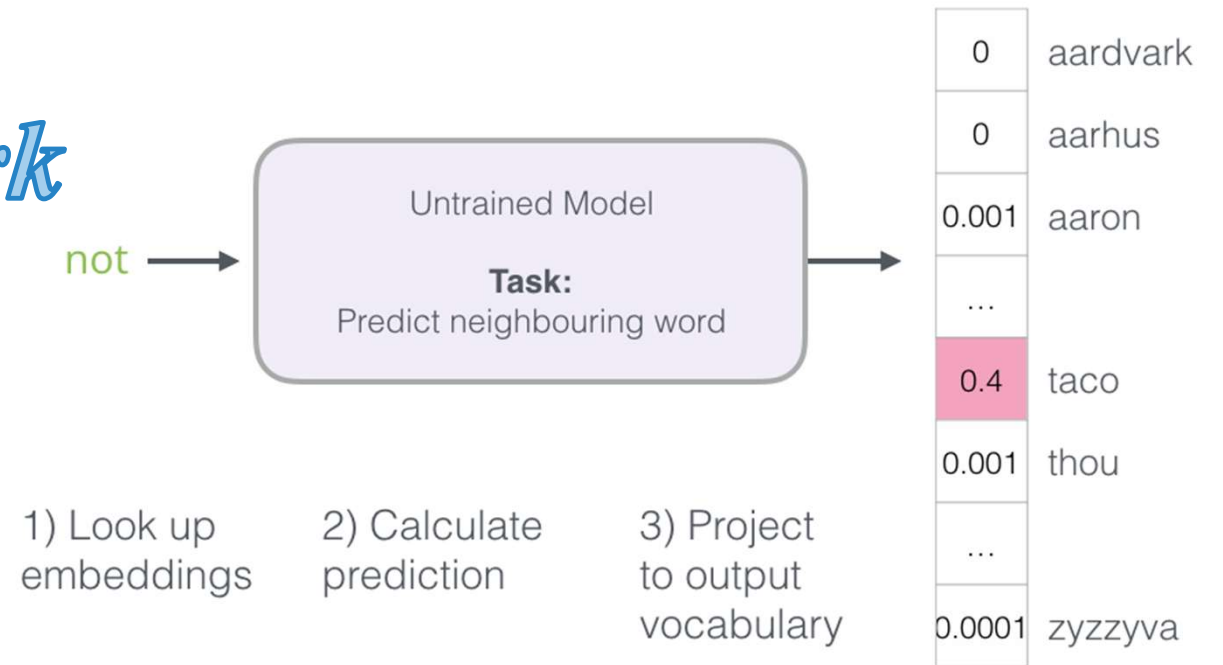
thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

- ✓ Words get their **embeddings** by us looking at which other **words they tend to appear next to**. The mechanics of that is that
 - We get **a lot of text data** (say, all Wikipedia articles, for example). then
 - We have **a window** (say, of three words) that we slide against all of that text.
 - The **sliding window generates training samples** for our model **(self-supervised!!!)**
- ✓ A sample dataset that we use to train a model.
 - *“Thou shalt not make a machine in the likeness of a human mind” ~Dune*

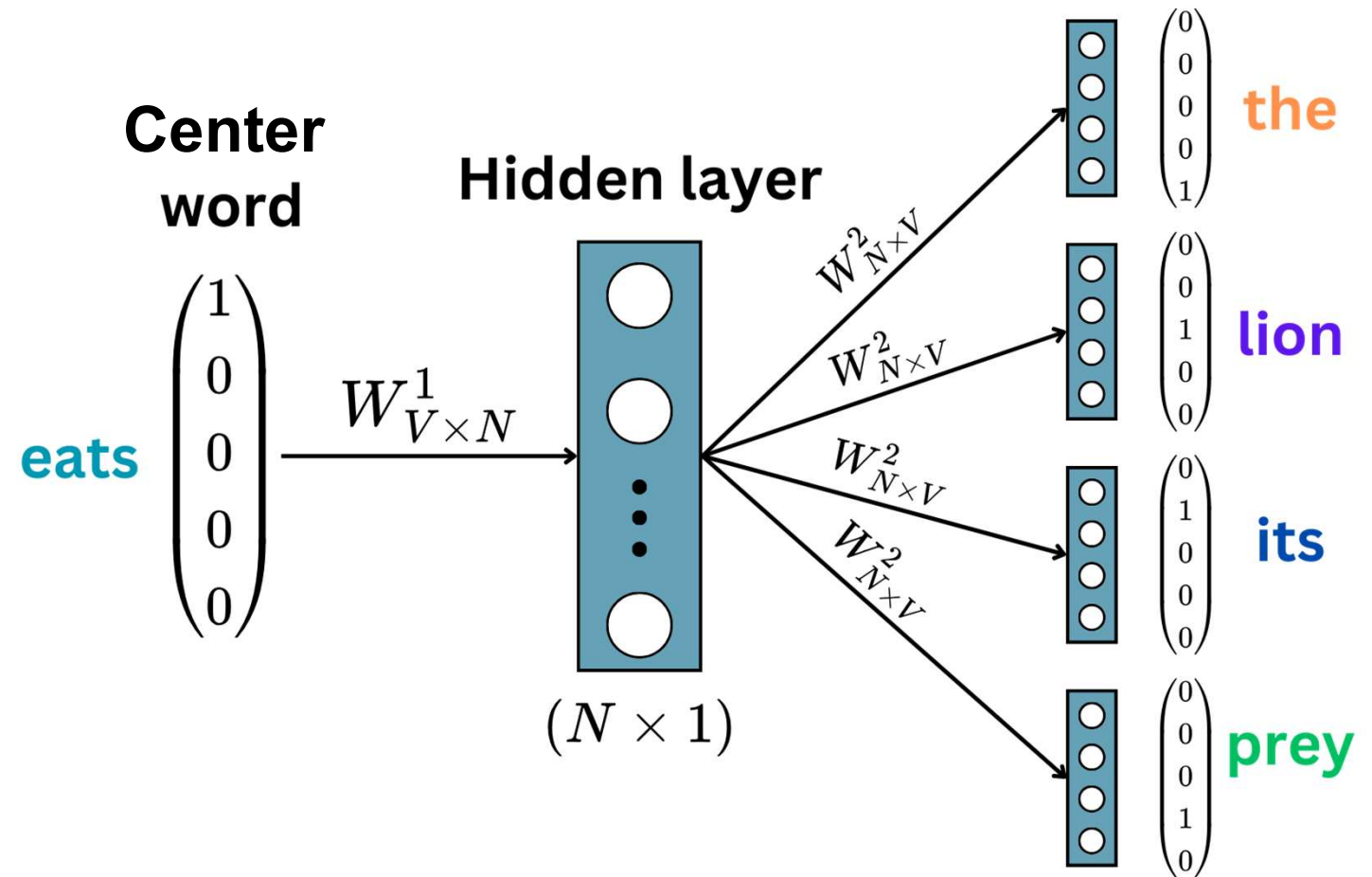
High-level neural network

-  Gathering training dataset without explicit supervision done!
- Next, train a **basic neural language model** that **predicts** the **neighboring word**.



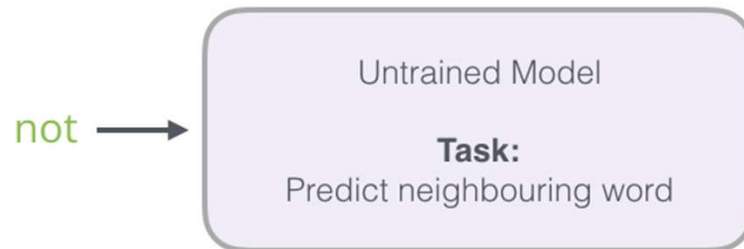
More details on the NN

It begins by retrieving the **embedding of the center word**, which is then used in the hidden layer. This is followed by **producing a $V \times 1$ vector in the output layer**.



Seems simple & easy! BUT...

- ⚠ The **third step** is very **expensive** from a computational point of view – especially knowing that we will do it once for every training sample in our dataset (easily tens of millions of times).

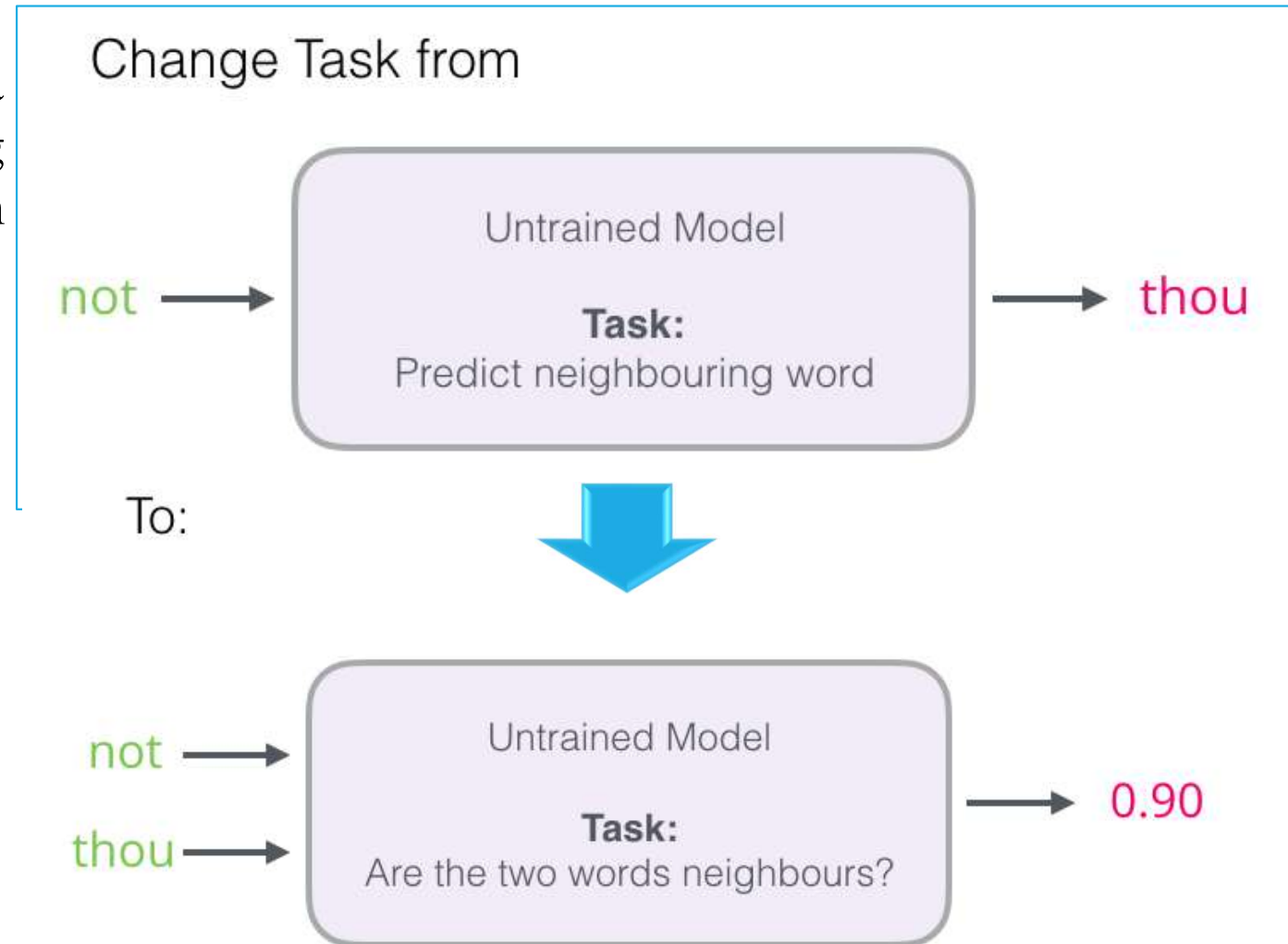


1) Look up embeddings

2) Calculate prediction

3) Project to output vocabulary

[Computationally Intensive]



This simple switch changes the model we need from a neural network, to a logistic regression model

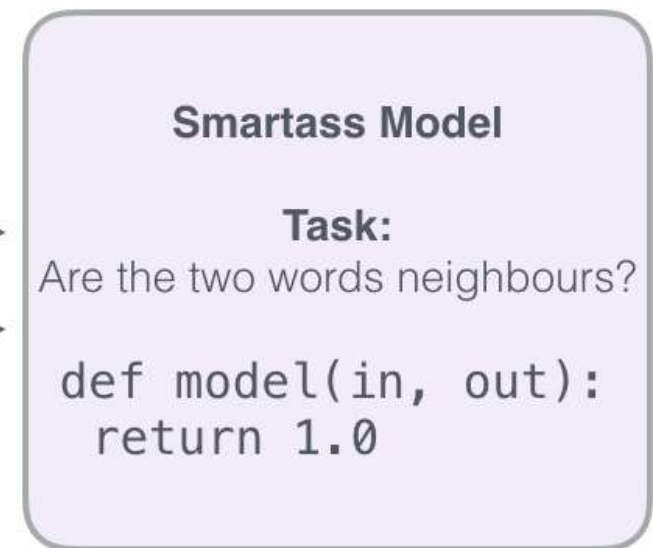
How to apply this change?

- This switch requires we **switch the structure of our dataset** – the **label is now a new column with values 0 or 1**. They will be **all 1** since all the words we added are neighbors.
- This can now be computed at **blazing speed** – processing millions of examples in minutes.
- **! !** But there's one loophole we need to close. **! !**
- If all of our examples are **positive (target: 1)**, we open ourself to the possibility of a **smartass model that always returns 1** – achieving 100% accuracy, but **learning nothing** and generating garbage embeddings.

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

not →
thou →



Skipgram with Negative Sampling (SGNS)

- To address this, we need to introduce *negative samples* to our dataset – samples of words that are not neighbors. Our model needs to return 0 for those samples.

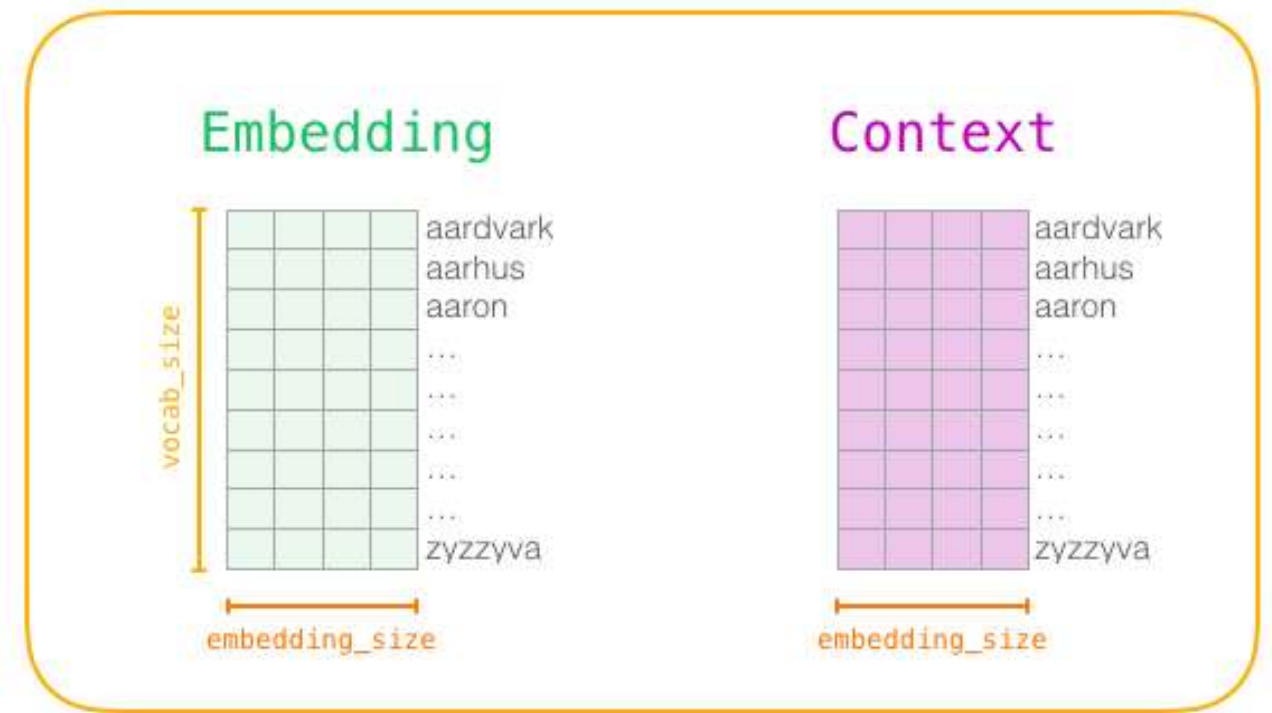
input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	make	1

Pick randomly from vocabulary
(random sampling)

Word	Count	Probability
aardvark		
aarhus		
aaron		
taco		
thou		
zyzzyva		

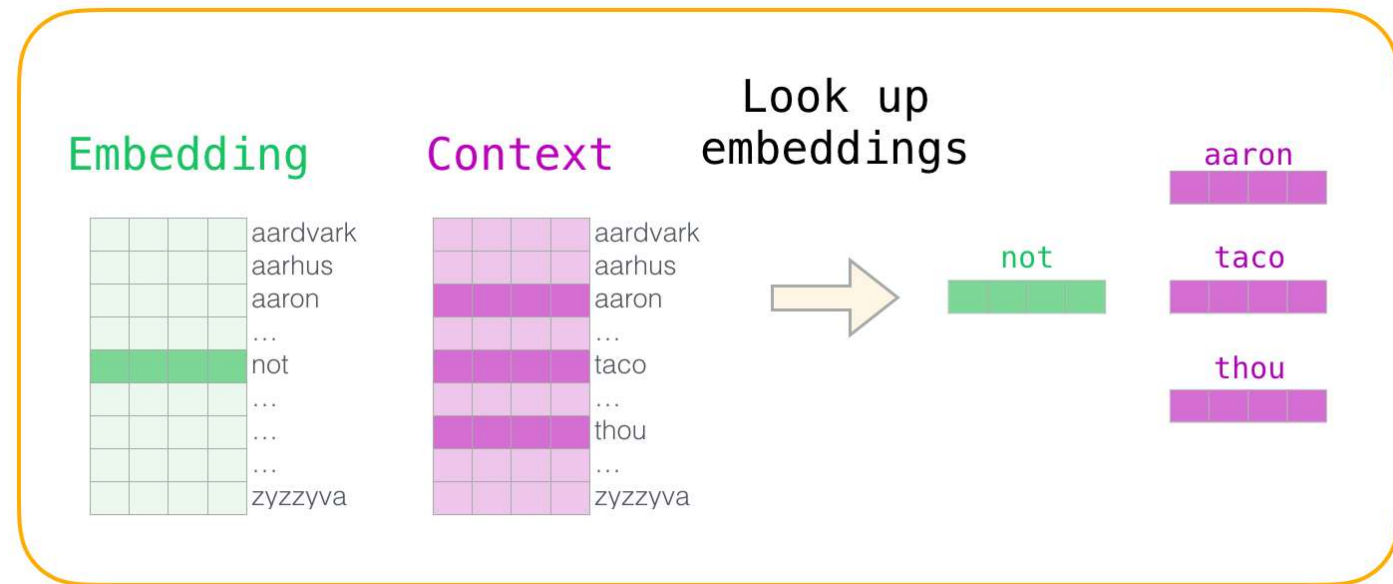
Training Process

- Before the training process starts, we determine the size of our vocabulary (we'll call this **vocab_size**, think of it as, say, 10,000)
- At the start of the training phase, we create **two matrices** – an **Embedding** matrix and a **Context** matrix.
- These two matrices have an embedding for each word in our vocabulary (So **vocab_size** is one of their dimensions). The second dimension is how long we want each embedding to be (**embedding_size** – 300 is a common value)

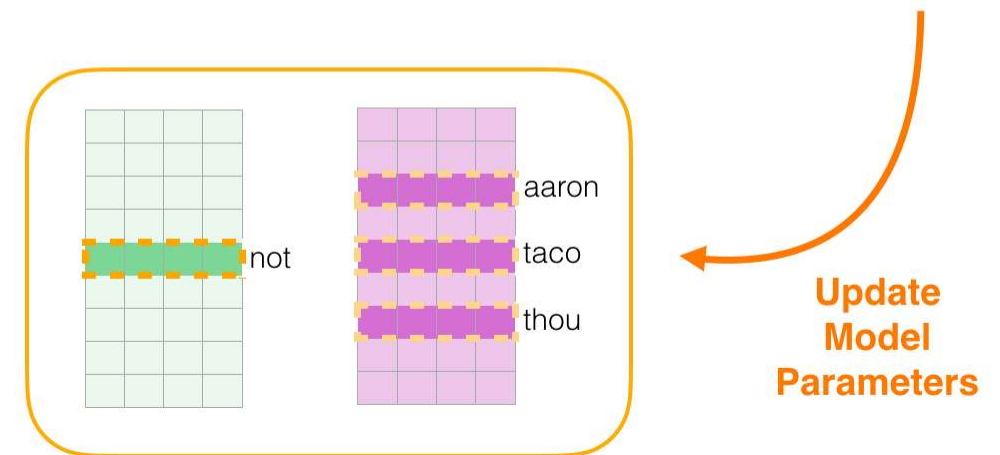


-cont-

- Initialize these matrices with random values.
- In each training step, we take one positive example and its associated negative examples.
- Now we have four words:
 - the input word **not**
 - output/context words: **thou** (the actual neighbor), **aaron**, and **taco** (the negative examples).
- We proceed to look up their embeddings:
 - For the **input word**, we look in the **Embedding** matrix.
 - For the **context words**, we look in the **Context** matrix (even though both matrices have an embedding for every word in our vocabulary)



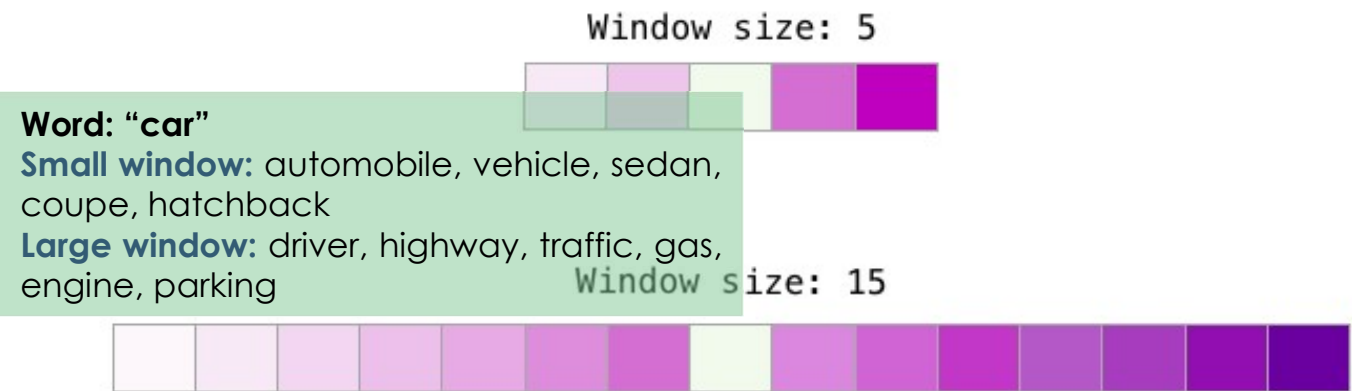
input word	output word	target	input • output	sigmoid()	Error
not	thou	1	0.2	0.55	0.45
not	aaron	0	-1.11	0.25	-0.25
not	taco	0	0.74	0.68	-0.68



Window Size and Number of Negative Samples

✓ Window Size:

- *smaller window sizes* (2-15) lead to embeddings where high similarity scores between two embeddings indicates that the words are *interchangeable*
- *Larger window sizes* (15-50, or even more) lead to embeddings where similarity is more indicative of *relatedness* of the words.



✓ # of Negative samples

- The original paper prescribes 5-20 as being a *good number of negative samples*.

Negative samples: 2

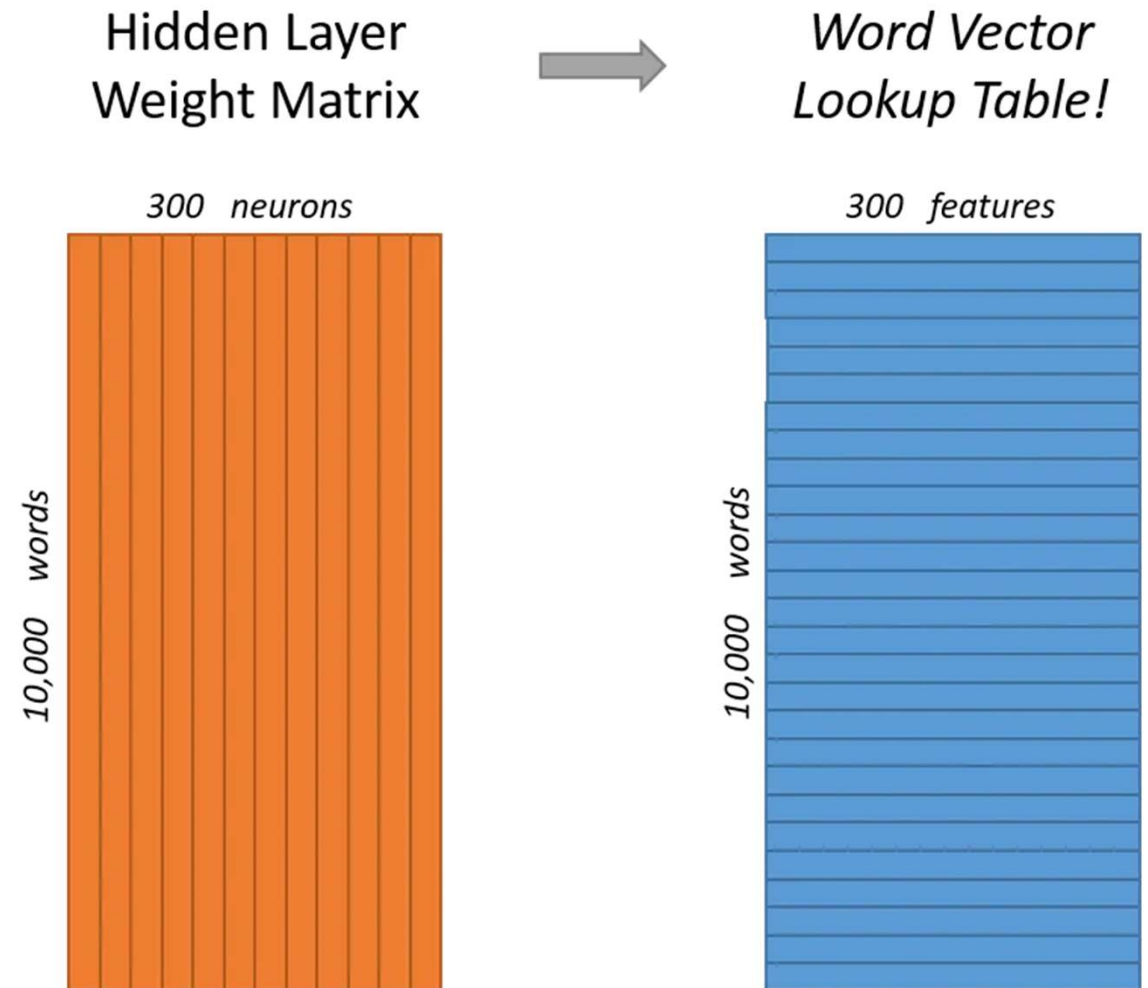
input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0

Negative samples: 5

input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0
make	finglonger	0
make	plumbus	0
make	mango	0

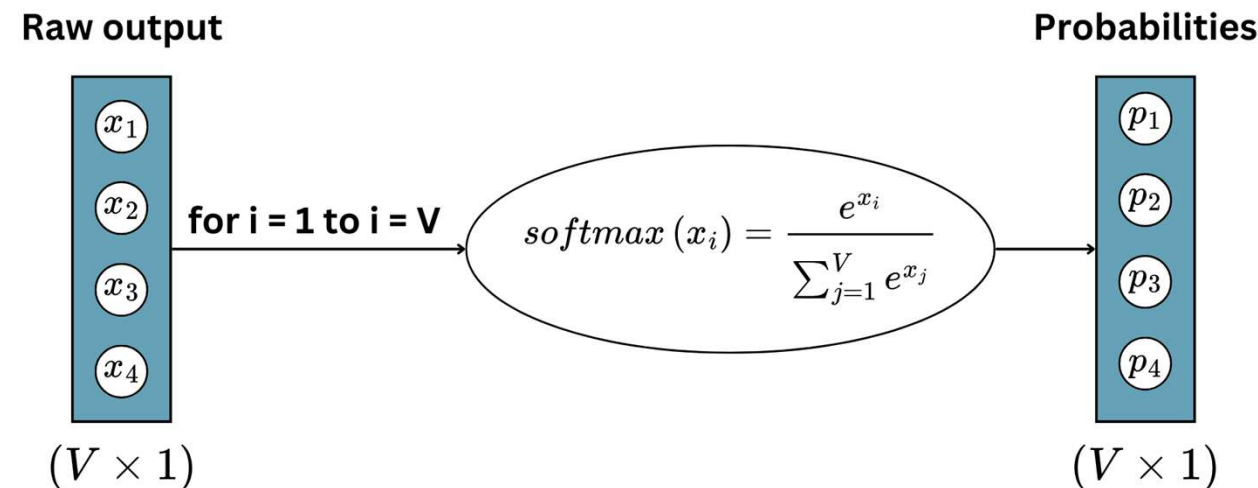
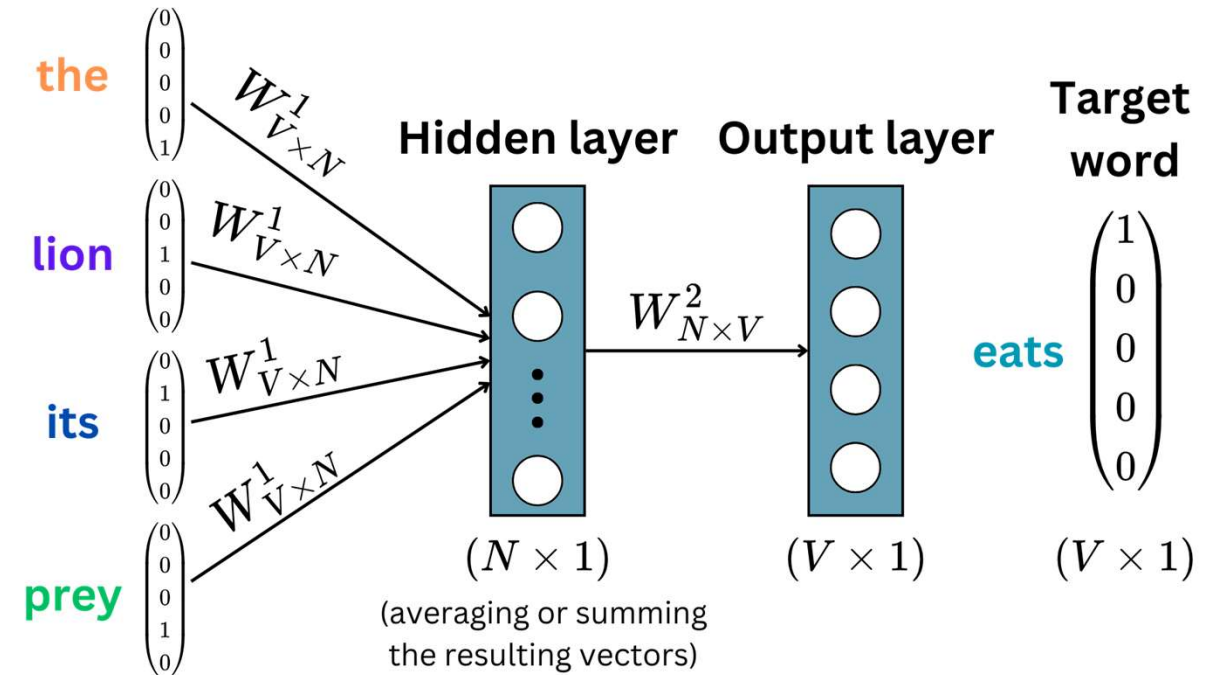
If still not clear...

- We said that Word2Vec is a group of models that tries to **represent each word** in a large text as a **vector** in a N-dimensional space.
- But where is this vector? 🤔
 - We used the **neural network** to find **some probabilities** but what we really care of is **not its output** but instead its **weights**.
 - In fact the **weights of the hidden layer** can be seen as the dimensions of the word vector where **each row represents a word**

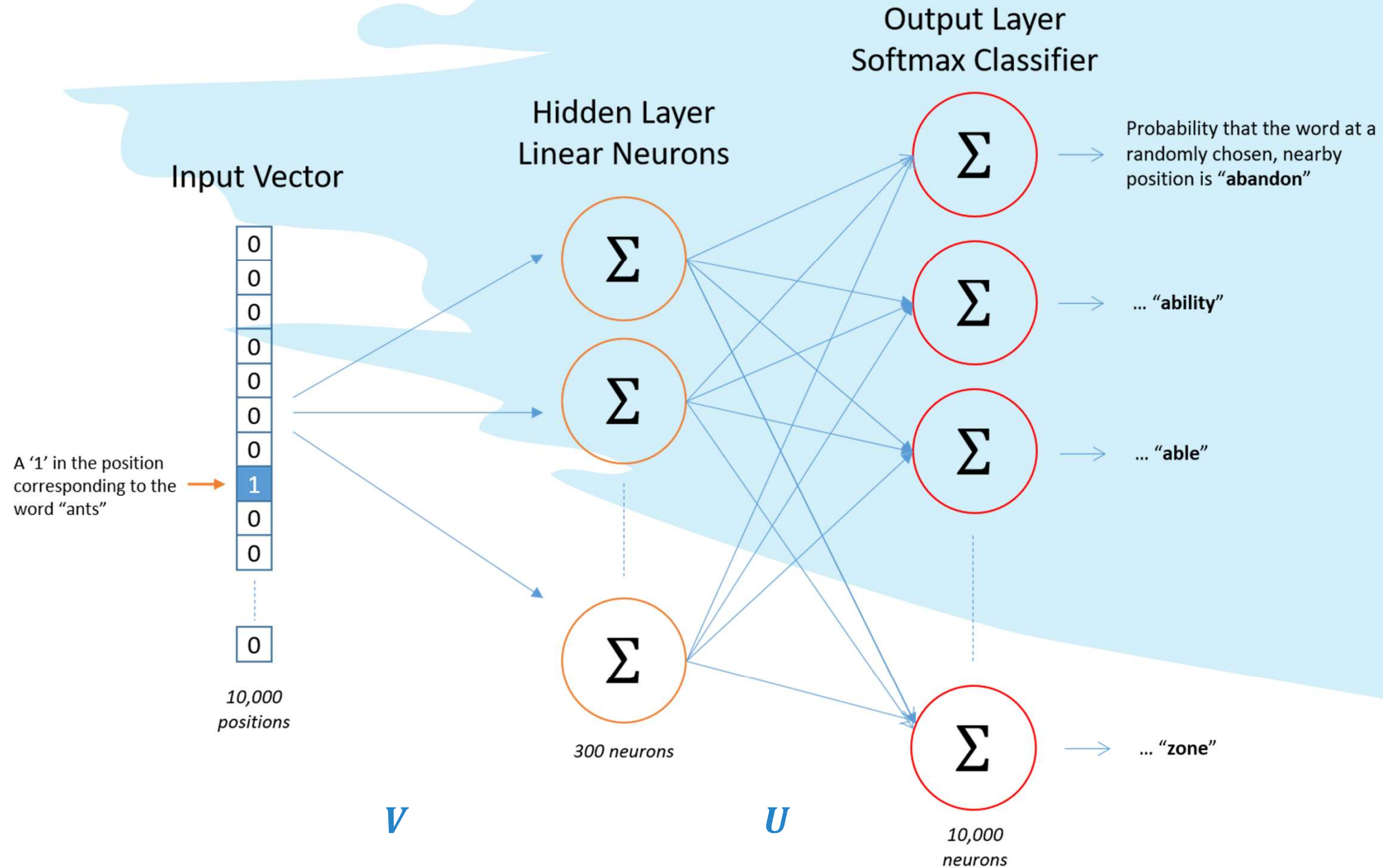


How about CBOW?

- CBOW treats **context** as a **bag** — **ignores word order**
- **Step-by-step:**
 - **Input:** a set (or window) of context words around a target.
 - Each context word is represented as a one-hot vector.
 - These vectors are mapped to embeddings via a shared embedding matrix V .
 - The embeddings of the context words are **averaged** (or summed).
 - The average context vector is multiplied by another matrix U to produce scores (logits) for all vocab words.
 - A softmax over logits gives a probability distribution for the center word.
 - **Cross-entropy loss** compares this distribution to the true center word (one-hot)



Mathematical Details of Skipgram



Quick Notes

Intuition:

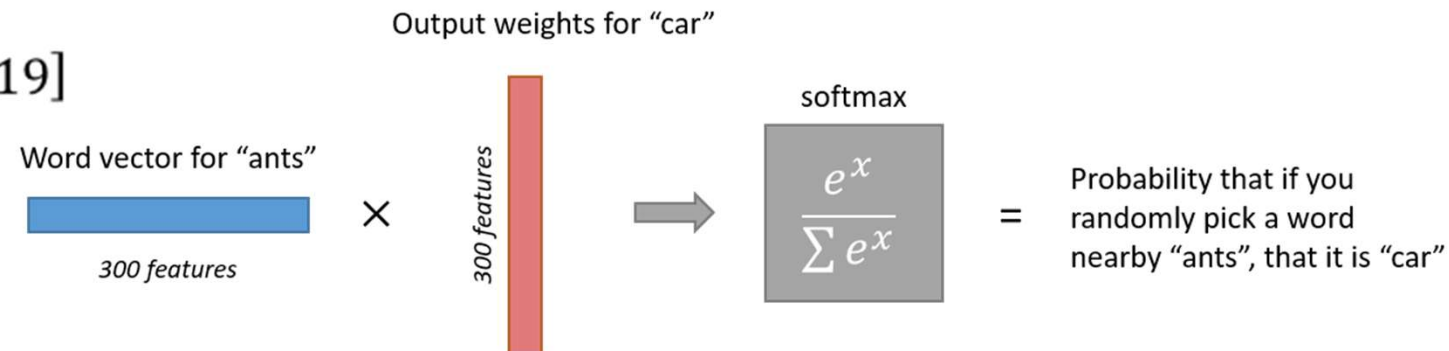
- ✓ If two different words have very similar “contexts” (that is, what words are likely to appear around them), then our model needs to output very similar results for these two words.
- ✓ One way for the network to output similar context predictions for these two words is if *the word vectors are similar*.
- ✓ So, if two words have similar contexts, then our network is **motivated** to learn similar word vectors for these two words!

- That one-hot vector is almost all zeros... what’s the effect of that?
- If you multiply a 1 x 10,000 one-hot vector by a 10,000 x 300 matrix, it will effectively just select the matrix row corresponding to the “1”

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

- This means that the hidden layer of this model is really just operating as a lookup table.
- The output of the hidden layer is just the “word vector” for the input word.

- Each output neuron has a weight vector which it multiplies against the word vector from the hidden layer, then it applies the function $\exp(x)$ to the result. Finally, in order to get the outputs to sum up to 1, we divide this result by the sum of the results from all 10,000 output nodes.



Note for Amirhossein K.

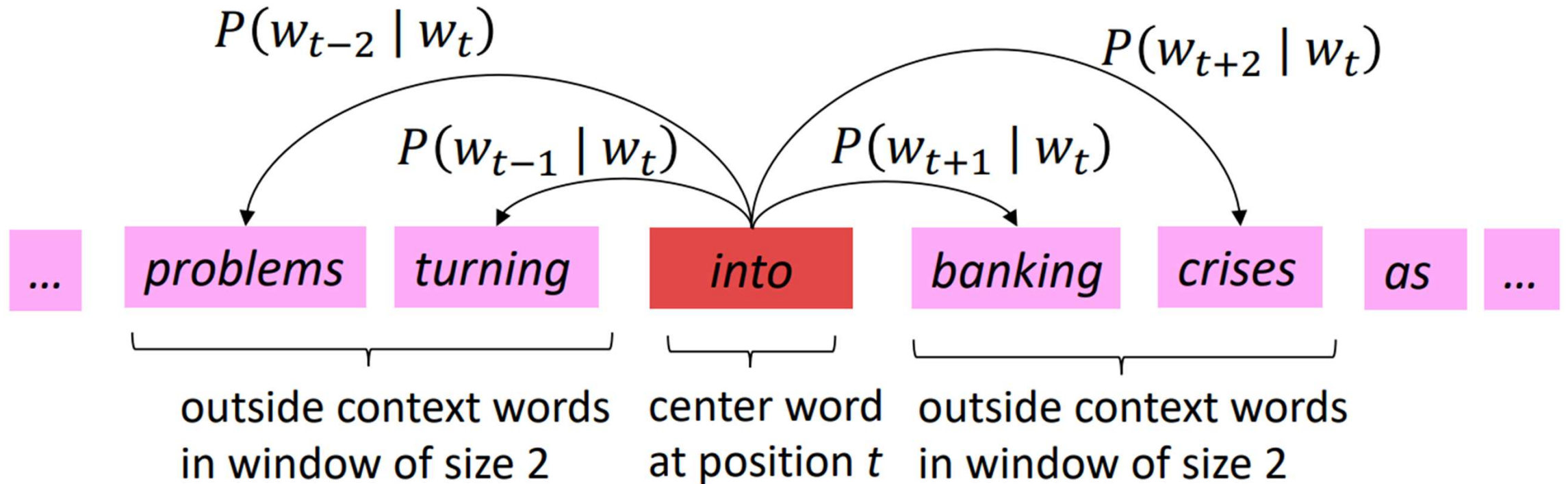
- ⚠ This can also handle **stemming** for you – the network will likely learn similar word vectors for the words “ant” and “ants” because these should have similar contexts.

Assumptions & Notations

- we have a finite vocabulary $\mathcal{V}$.
- Let C, O be random variables representing pair of $C \in \mathcal{V}$ (a center word), and $O \in \mathcal{V}$ (an outside word, appearing in the context of the center word).
- Let $U \in \mathbb{R}^{|\mathcal{V}| \times d}$ and $V \in \mathbb{R}^{|\mathcal{V}| \times d}$.
- Note that each word in $\mathcal{V}$ is associated with a single row of U and one of V .
- The word2vec model is a probabilistic model specified as follows, where u_w refers to the row of U corresponding to word $w \in \mathcal{V}$ (and likewise for V):

$$p_{U,V}(o \mid c) = \frac{e^{u_o^T v_c}}{\sum_{w \in \mathcal{V}} e^{u_w^T v_c}}$$

Example windows and process for computing



Objective function

- For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t !. Data likelihood:

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$

θ is all variables to be optimized

- The objective function $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

Now, how do we do the minimization?

- we'll write out the partial gradient of the loss with respect to the parameter v_c for a single instance of a center word c :
- $\nabla_{v_c} \log p_{U,V}(o | c) = \nabla_{v_c} \log \frac{e^{u_o^T v_c}}{\sum_{w \in \mathcal{V}} e^{u_w^T v_c}} = \underbrace{\nabla_{v_c} \log e^{u_o^T v_c}}_{\text{teal}} - \underbrace{\nabla_{v_c} \log \sum_{w \in \mathcal{V}} e^{u_w^T v_c}}_{\text{blue}}$
- $\nabla_{v_c} \log e^{u_o^T v_c} = u_o$
- $\nabla_{v_c} \log \sum_{w \in \mathcal{V}} e^{u_w^T v_c} = \frac{1}{\sum_{w \in \mathcal{V}} e^{u_w^T v_c}} \sum_{w \in \mathcal{V}} \nabla_{v_c} e^{u_w^T v_c}$
$$= \frac{1}{\sum_{w \in \mathcal{V}} e^{u_w^T v_c}} \sum_{w \in \mathcal{V}} e^{u_w^T v_c} u_w = \sum_{x \in \mathcal{V}} \frac{e^{u_x^T v_c}}{\sum_{w \in \mathcal{V}} e^{u_w^T v_c}} u_x = \sum_{x \in \mathcal{V}} P_{U,V}(x|c) u_x = E[u_x]$$
- $\nabla_{v_c} \log p_{U,V}(o | c) = u_o - E[u_x] = \text{"observed"} - \text{"expected"}$
- The v_c vector is updated to be “more like” the word vector that was actually observed than the word vector it expected

Skipgram with negative sampling

- ✓ For a given word, it is cheap to compute the unnormalized score $\exp(u_c^\top v_o)$.
- ✓ However, it is expensive to compute the sum of scores for all words in the denominator
- ✓ From a probabilistic perspective, the partition function guarantees a probability by normalizing the scores to sum to 1.
- ✓ From a learning perspective, the partition function “pushes down” on all the words other than the observed words.
- ✓ The numerator of this equation encourages the model to make u_o more like v_c ; the denominator encourages all the other u_w for $w \neq o$ less like v_c .
- ✓ The intuition of negative sampling is that we don't need to push down on all the u_w all the time, since that's where most of the cost comes from.
- ✓ skip-gram with negative sampling (SGNS) objective ends up being a bit more different:

$$p_{u,v}(o|c) = \frac{\exp u_o^\top v_c}{\sum_{w \in \mathcal{V}} \exp u_w^\top v_c}$$

Affinity of word c for context o

Partition function, or, normalization

$$\log \sigma(u_o^\top v_c) + \sum_{\ell=1}^k \left[\log \sigma(-u_\ell^\top v_c) \right]$$

Affinity of word c for context o

Push down everything else in expectation (replacement for partition fn)

Selecting negative samples

- The “negative samples” (that is, the 5 output words that we’ll train to output 0) are selected using a “unigram distribution”, where more frequent words are more likely to be selected as negative samples.
- The authors state in their paper that they tried a number of variations on this equation, and the one which performed best was to raise the word counts to the 3/4 power.
- Compared to the simpler equation:
 - increase the probability for less frequent words
 - decrease the probability for more frequent words.

$$P(w_i) = \frac{f(w_i)}{\sum_{j=0}^n f(w_j)}$$

$$P_{0.75}(w_i) = \frac{f(w_i)^{\frac{3}{4}}}{\sum_{j=0}^n f(w_j)^{\frac{3}{4}}}$$

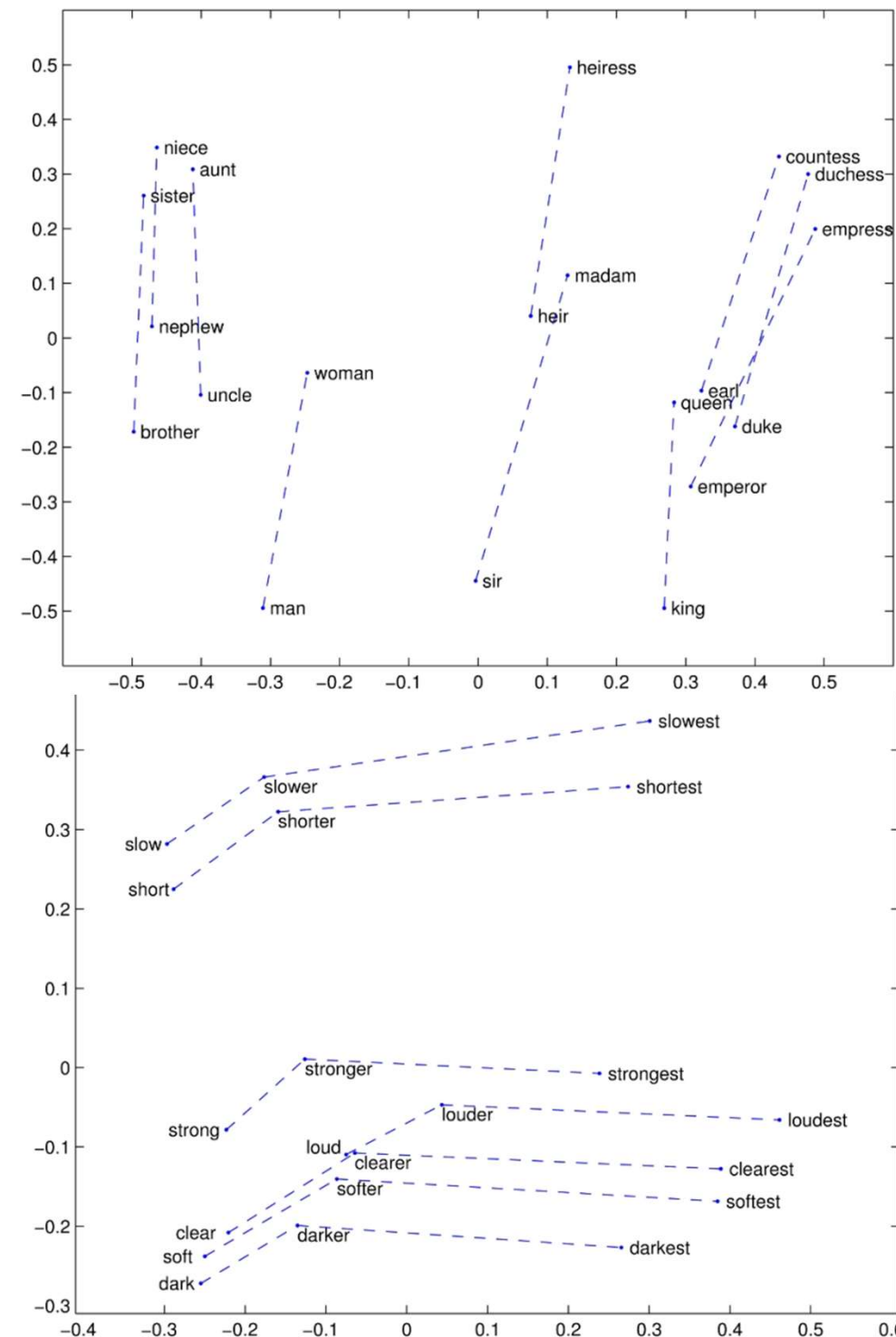
Example:

$$P_{0.75}(a) = \frac{0.99^{0.75}}{0.99^{0.75} + 0.01^{0.75}} = 0.97$$

$$P_{0.75}(b) = \frac{0.01^{0.75}}{0.99^{0.75} + 0.01^{0.75}} = 0.03$$

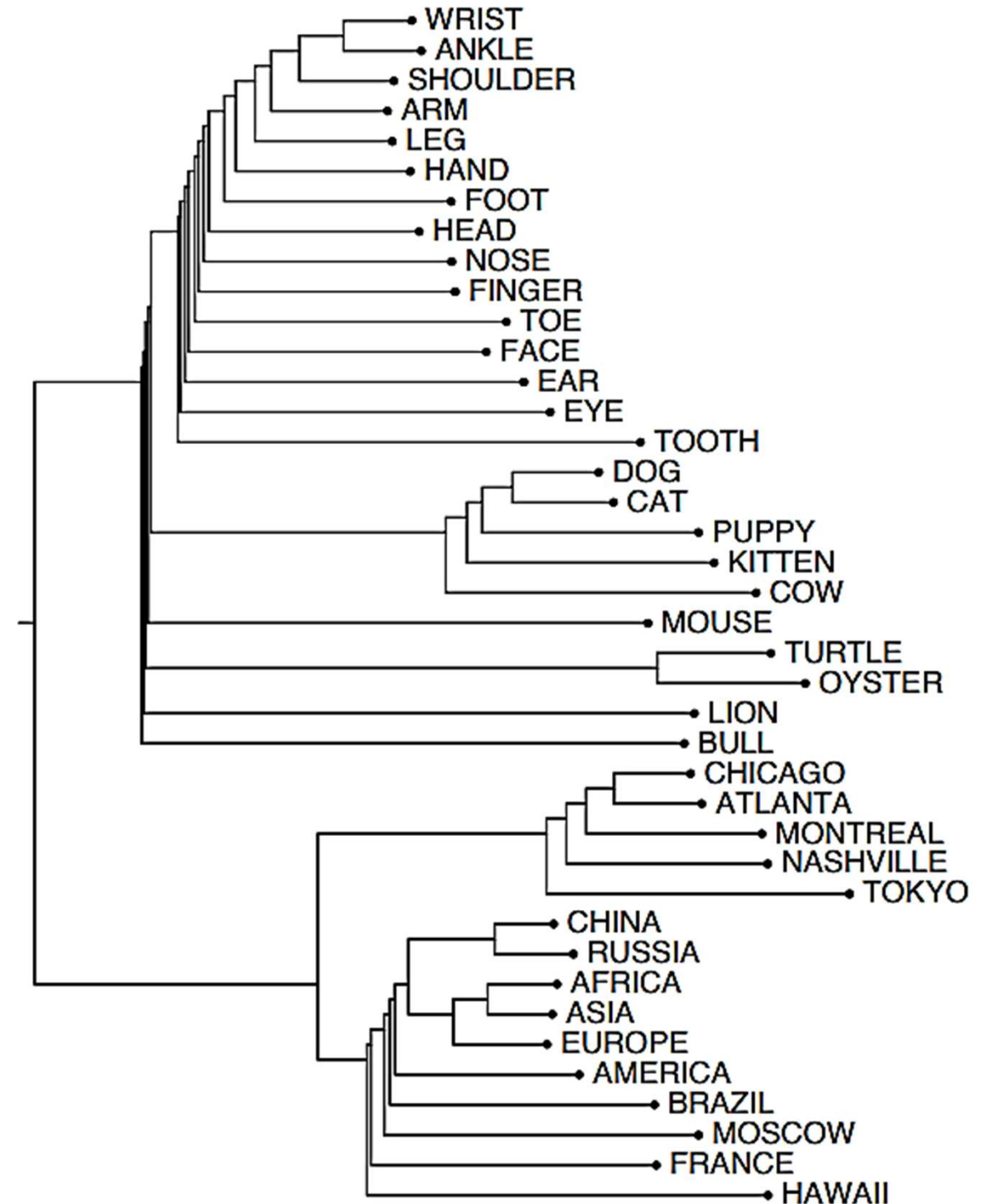
Semantic properties of embeddings

- The result of the expression
 - $\text{king} - \text{man} + \text{woman}$ is a vector close to **queen**.
 - $\text{Paris} - \text{France} + \text{Italy}$ results in a vector that is close to **Rome**
- Embedding model thus seems to be extracting representations of relations like
 - *MALE-FEMALE*,
 - *CAPITAL-CITY-OF*,
 - *COMPARATIVE/SUPERLATIVE*



Visualizing Embeddings

- ✓ The simplest way to visualize the meaning of a word w embedded in a space is to list the most similar words to w by sorting the vectors for all words in the vocabulary by their cosine with the vector for w .
- ✓ Another visualization method is to use a clustering algorithm to show a hierarchical representation of which words are similar to others in the embedding space
- ✓ The most common visualization method, however, is to project the 100 dimensions of a word down into 2 dimensions using t-sne



Word senses and word sense ambiguity

Most words have **lots of meanings!**

Especially common words (bank, seal, rock, etc)

Especially words that have existed for a long time

Example: pike (fish, weapon, road, etc)

Does **one vector** capture **all these meanings** or do we have a mess?